

Progetto Sistemi Operativi 2023-2024 - Reattore

Gabriele Buoso
Università degli Studi di Torino
1068499

Andrea Driza
Università degli Studi di Torino
1042190

Mattia Nada
Università degli Studi di Torino
1043526

1. Informazioni preliminari per la comprensione

1.1. Master

Master è il metodo principale del progetto ed è il responsabile di:

1. inizializzazione e avvio simulazione
2. stampa, ad ogni secondo, delle statistiche relative ed assolute
3. terminazione della simulazione di tutti i processi
4. terminazione dei processi
5. deallocare le risorse di IPC.

1.2. Atomo

Atomo è la fonte di energia del reattore e dopo aver inizializzato le sue strutture e impostato i segnali, inizia la simulazione (solo se è un atomo creato direttamente da Master) identificandosi su una coda di messaggi con mtype = 1. Durante l'esecuzione non fa altro che aspettare la ricezione del segnale SIGUSR1, una volta svegliato dal segnale mandato dal processo Attivatore continua la computazione creando due nuovi atomi solo se necessario.

	argv[0]	argv[1]	argv[2]	argv[3]	argv[4]
Atomo	NUM_ATOMICO	msgid	shmid	semid	NULL

Table 1. Parametri passati da Master

	argv[0]	argv[1]	argv[2]	argv[3]
Atomo	NUM_ATOMICO	msgid	shmid	NULL

Table 2. Parametri passati da Atomo

1.3. Alimentazione

Alimentazione, dopo aver inizializzato le sue strutture e aver iniziato anch'esso la simulazione, eseguirà un loop infinito, nel quale ogni STEP_ALIMENTAZIONE nanosecondi crea nuovi atomi. Ad ogni iterazione esegue una fork() creando due nuovi processi Atomo, passandogli gli opportuni parametri.

	argv[0]	argv[1]	argv[2]	argv[3]
Alimentazione	semid	msgid	shmid	NULL

Table 3. Parametri passati da Master

1.4. Attivatore

Attivatore dopo aver inizializzato le sue strutture ed aver iniziato la simulazione, entra in un loop infinito dove ogni STEP_ATTIVATORE millisecondi preleva il PID dei processi Atomo preleva i messaggi con mtype = 1 dalla coda e invia ai PID letti il segnale di attivarsi SIGUSR1.

	argv[0]	argv[1]	argv[2]	argv[3]
Attivatore	semid	msgid	shmid	NULL

Table 4. Parametri passati da Master

1.5. Inibitore

Il processo Inibitore può essere attivato a discrezione dell'utente all'inizio della simulazione e può essere acceso o spento con la sequenza CTRL + C da tastiera. Si occupa di regolare la simulazione evitando le terminazioni Meltdown e Explode. Dopo aver inizializzato le sue strutture e impostato i relativi handler ai segnali, entra in un loop infinito dove, dopo aver monitorato il livello di energia all'interno

del reattore, assorbe parte dell'energia della scissione oppure se il reattore sta per esplodere, ordina al processo Attivatore di non far scindere i processi Atomo.

	argv[0]	argv[1]	argv[2]	argv[3]
Inibitore	semid	msgid	shmid	NULL

Table 5. Parametri passati da Master

2. Inizializzazione Processi e Avvio simulazione

Il processo Master crea un'unica coda di messaggi e un set di due semafori, utili per l'inizializzazione e l'avvio della simulazione. e due porzioni di memoria condivise. Alloca anche una porzione di memoria condivisa tra tutti i processi corrispondente ad array di tipo struct stats_scissioni, l'indice 0 rappresenta le statistiche assolute e l'indice 1 quelle relative. Il processo Master, successivamente, si identifica sulla coda tramite il proprio PID con mtype = 2 in modo tale che il primo processo che rilevi il fallimento di una fork() possa comunicargli tramite il segnale SIGTERM di terminare la simulazione per Meltdown. Master alloca anche un'altra porzione di memoria, condivisa solo con Attivatore e Inibitore tramite due messaggi, rispettivamente mtype = 10 e mtype = 11. Quest ultima è di tipo struct stats_inibitore e contiene informazioni relative esclusivamente allo stato di Inibitore, come una variabile che indica se Inibitore è spento e acceso, il numero e il tipo di operazioni fatte. Master dopo aver creato i processi con gli opportuni parametri, tenta di fare un'operazione bloccante sul primo semaforo della quantità $-(N_ATOMI_INIT + 3)$, possibile solo quando tutti i processi iniziali hanno finito la loro rispettiva simulazione e incrementato di uno il valore del primo semaforo. Tutti i processi quindi attendono di poter fare un'operazione di -1 sul secondo semaforo di inizio simulazione, possibile solo quando master, rilevata la fine inizializzazione, lo incrementa della quantità $N_ATOMI_INIT + 3$. Se tutti i processi iniziano correttamente la simulazione inizia.

3. Simulazione funzionamento

All'avvio della simulazione Master entra in un loop di SIM_DURATION iterazioni, nel quale ad ognuna esegue una sleep(1) e, nel caso non fosse avvenuta nessuna terminazione, stampa le statistiche relative e assolute; in caso contrario salva in una variabile intera la causa della terminazione e imposta una flag a 0 in modo tale da uscire dal ciclo all'iterazione successiva. Il processo Alimentazione durante tutta la simulazione crea N_NUOVI_ATOMI ogni STEP_ALIMENTAZIONE, i quali non eseguiranno più operazioni sui semafori, siccome non sono Atomi creati da

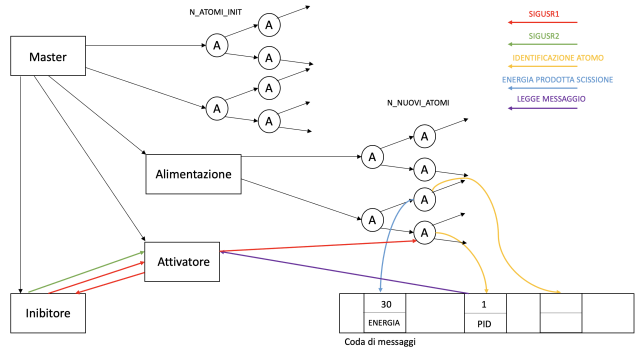


Figure 1. Sincronizzazione processi

Master (il controllo è fatto su argv[3], vedi Table 2.). I processi Atomo si identificano sulla coda di messaggi tramite il loro PID (della quale Attivatore si servirà) e successivamente attendono il segnale di scindersi SIGUSR1 da parte di Attivatore. La scissione, dunque, è dettata da Attivatore che durante l'esecuzione del suo loop infinito comunica costantemente con Inibitore. I due processi sopracitati si sincronizzano con un semplice handshake ad ogni iterazione di Attivatore. Quest ultimo dopo aver aspettato STEP_ATTIVATORE microsecondi con una usleep(), manda un segnale ad Inibitore di tipo SIGUSR1 e si mette in coda di ready con pause(). Inibitore, anch'esso in un loop infinito ma addormentato su una pause(), riceve il segnale svegliandosi. Da questo in punto poi la computazione si dirama:

1. Nel caso in cui flag_inib sia 0 (Inibitore spento), dopo aver scartato dalla coda i messaggi contenenti l'energia liberata dalla scissione, Inibitore manda un segnale di tipo SIGUSR1, che sveglia Attivatore da una pause() e gestisce il relativo handler contenente il codice per far scindere gli Atomi
2. Nel caso in cui flag_inib sia 1 (Inibitore acceso), calcola il livello di energia sul totale e di conseguenza ci sono altre due rami di computazione:
 - (a) Se il livello di energia percentuale (calcolata come Energia prodotta netta / ENERGY_EXPLODE_THRESHOLD) è inferiore SOGLIA_MASSIMA, non c'è pericolo di Explode o Meltdown. Dopo aver letto i messaggi dalla coda mandati da Atomo contenenti l'energia liberata dalle scissioni, ne assorbe il 20% e procede con il punto 1).
 - (b) Altrimenti Inibitore manda un segnale di tipo SIGUSR2 ad Attivatore. L'handler di questo segnale non contiene alcun codice, infatti Attivatore non deve ordinare agli Atomi di forkare.

Un generico Atomo quindi, addormentato su una `pause()`, viene svegliato e:

- Se il `NUM_ATOMICO` assegnatogli dal padre (Master, Alimentazione o altro Atomo) è sufficiente, scinde con una `fork()`, creando due nuovi Atomi dividendo il proprio `NUM_ATOMICO` in parti uguali ai due figli.
- Se il `NUM_ATOMICO` assegnatogli non è sufficiente, viene calcolata come scoria.

4. Terminazioni

La terminazione per Timeout corrisponde alla condizione di uscita del loop di Master, ovvero quando il contatore raggiunge il valore di `SIM_DURATION`. Le terminazioni per Explode e Blackout sono controllate da Master nel loop della simulazione, eseguendo semplicemente un calcolo dell'energia prodotta al netto, servendosi dei dati sulla memoria condivisa delle statistiche. Diversa è la terminazione per Meltdown, che può essere rilevata da Master (quando inizializza i processi), Alimentazione e Atomo. Come già spiegato nella sez. II, il primo processo che rileva il fallimento di una `fork()`, preleva il messaggio (`mtype = 2`) di identificazione di Master e gli invia il segnale `SIGUSR2`. In tutti i casi Master, dopo aver rilevato la terminazione, deve terminare tutti i processi attivi della simulazione, inviando quindi un segnale di tipo `SIGTERM` a tutti i processi con egual group PID (tutti sono figli di Master e avranno group PID uguale).

5. Test Terminazioni

	Timeout	Explode	Blackout	Meltdown
SIM_DURATION	10	20	15	10
N_ATOMLINIT	30	50	5	100
N_NUOVLATOMI	5	5	2	60
ENERGY_EXPLODE_THRESHOLD	20000	10000	10000	30000
ENERGY_DEMAND	300	150	300	100

Table 6. Configurazioni